

ARTH-AI

NEON / SUPABASE DEPLOYMENT GUIDE

Exact setup, connection, deployment, verification, and cost-safety steps.

Recommendation

Use Neon first for the database. It is the simplest fit for this project because the backend already uses normal PostgreSQL through psycopg and does not require Supabase-specific SDKs.

Use this architecture:

- GitHub: source code
- Vercel or Azure Static Web Apps: frontend
- Azure Container Apps: backend
- Neon or Supabase: PostgreSQL database
- Google Gemini: optional chat enhancement

You do not need Azure PostgreSQL.

Important Cost Warning

No provider can guarantee zero cost. Free tiers and student credits have limits and can change. Before creating anything:

1. Confirm the provider shows a free plan or available credit.
2. Set a budget alert where available.
3. Do not enable paid compute, high availability, backups, or large storage.
4. Delete the project after the demo if you no longer need it.

Accounts and Services

Create or verify these accounts:

1. GitHub account with access to <https://github.com/snehpatel05/HackOut-26>.
2. Neon account at <https://console.neon.tech>, or Supabase account at <https://supabase.com/dashboard>.
3. Azure account only if deploying the backend with Azure Container Apps.
4. Vercel account at <https://vercel.com>, or Azure Static Web Apps for the frontend.
5. Google AI Studio only if Gemini chat is required.

Option A: Neon Database

Create the database

1. Open <https://console.neon.tech>.
2. Create a project.
3. Project name: arthai.
4. Select the closest region.

5. Use the free plan if it is available to your account.
6. Open the project dashboard and choose **Connect**.
7. Select the PostgreSQL connection string.
8. Copy the pooled or direct connection string.

Neon may provide a URL beginning with `postgresql://`. The application driver needs the SQLAlchemy `psycopg` prefix. Change only the beginning:

to:

Keep the rest of the URL unchanged.

Neon database environment variable

Use this exact backend variable:

Do not paste this value into frontend code. Do not commit it to GitHub.

Option B: Supabase Database

Create the database

1. Open <https://supabase.com/dashboard>.
2. Create a new project named `arthai`.
3. Choose a nearby region.
4. Create a strong database password and store it safely.
5. Wait for the project to finish provisioning.
6. Open **Project Settings -> Database**.
7. Find **Connection string**.
8. Choose the URI or SQLAlchemy-compatible connection option if shown.
9. Prefer the pooled connection string for serverless or frequently sleeping hosts.

If the Supabase string begins with `postgresql://`, change the prefix to:

Example:

Supabase may require URL-encoding special characters in the password. For example, `@` becomes `%40` and `#` becomes `%23`.

Do not use a Supabase anon key as `DATABASE_URL`. The backend needs the PostgreSQL connection string.

Test the Database Before Deployment

From PowerShell, inside backend, create a temporary `.env` file with the database URL:

Put this inside `.env`:

Do not commit `.env`.

Run migrations and seed data:

Start the backend:

In another PowerShell window, test:

Expected response:

Backend Deployment on Azure Container Apps

Use Azure Container Apps for the backend only. Keep the database on Neon or Supabase.

1. Create an Azure Container Apps environment.
2. Build the image from `backend/Dockerfile`.
3. Deploy the image with external ingress.
4. Set target port to 8000.
5. Use the same Azure region as your other services when practical.
6. Add the environment variables below in the Container App configuration.

Required backend variables:

Optional backend variables:

Generate a secret locally in PowerShell:

Do not put `DATABASE_URL`, `JWT_SECRET`, or `GEMINI_API_KEY` in GitHub files. Add them as Container App secrets or environment variables.

After deployment, copy the backend URL. It will look similar to:

Test:

Frontend Deployment

Vercel option

1. Open Vercel and import the GitHub repository.
2. Set the root directory to `frontend`.
3. Framework: Next.js.
4. Build command: `npm run build`.
5. Add this environment variable before deploying:
6. Deploy.

7. Copy the frontend URL.
8. Update backend CORS_ORIGINS to the exact frontend URL without a trailing slash.
9. Redeploy the backend.

Azure Static Web Apps option

1. Create an Azure Static Web App.
2. Connect the GitHub repository.
3. App location: /frontend.
4. Output location: .next if using the supported Next.js deployment flow.
5. Add this application setting:
6. Deploy.
7. Copy the generated frontend URL.
8. Set backend CORS_ORIGINS to that URL.

For the simplest deployment, Vercel is usually easier for this Next.js app. Azure can still host the backend and database connection remains external on Neon or Supabase.

Exact Production Values

Backend environment:

Frontend environment:

Replace every placeholder. Do not include angle brackets in the final values.

APIs Used by This Project

The frontend calls the backend routes:

External services required:

- Neon or Supabase PostgreSQL: required for hosted persistent data.
- Gemini API: optional; only used for richer chat responses.

External services not required by the current implementation:

- UPI API
- Bank account API
- DigiLocker API

- Aadhaar API
- PAN verification API
- Payment gateway
- WhatsApp API
- Credit bureau API
- External fraud API

Those features are mocked or implemented locally for the hackathon.

Deployment Verification

Run these checks in order:

1. Open the backend `/health` URL and confirm status and db are both ok.
2. Open the frontend URL.
3. Log in with phone 90000000001 and PIN 1234.
4. Confirm the dashboard loads.
5. Confirm transactions and offers appear.
6. Complete mock KYC if requested.
7. Open Alerts, Conversation, Explain, and Simulator.
8. Run one simulator transaction.
9. Check the backend logs for errors.
10. Confirm the browser developer console has no CORS or mixed-content errors.

Cost and Cleanup

For Neon or Supabase:

- Stay on the free plan while eligible.
- Do not enable paid add-ons.
- Set usage alerts if the provider supports them.
- Delete the project after the hackathon if it is no longer needed.

For Azure:

- Confirm the subscription is active.
- Set a budget alert.
- Use only the smallest Container App resources.
- Stop or delete unused Container Apps.
- Never assume student credit means unlimited free usage.

Final Deployment Checklist

- ☐ Database project created on Neon or Supabase.
- ☐ PostgreSQL URL changed to `postgresql+psycopg://`.
- ☐ Password special characters URL-encoded.
- ☐ Backend `DATABASE_URL` added as a secret.
- ☐ Strong `JWT_SECRET` added as a secret.

- ☐ Backend deployed on port 8000.
- ☐ /health returns {"status": "ok", "db": "ok"}.
- ☐ Frontend NEXT_PUBLIC_API_URL points to the hosted backend.
- ☐ Backend CORS_ORIGINS points to the hosted frontend.
- ☐ Login works with the demo account.
- ☐ No secret is committed to GitHub.
- ☐ Cost alerts are configured.
- ☐ Unused resources are deleted after the demo.